



香港中文大學 (深圳)

The Chinese University of Hong Kong

CSC 5003 Algorithm Art and Programming Practice Lecture 5: String Processing

Jingbang Chen

School of Data Science (SDS)

The Chinese University of Hong Kong, Shenzhen



String Matching

- ▶ Applications of string matching
- ▶ How to match strings?
 - Exact matching
 - Approximate matching (alignment)
- ▶ Exact matching
 - A brute force algorithm
 - The Knuth-Morris-Pratt (KMP) algorithm



Applications of string matching

► Document search

- Browsers
- Text editors
- Many tools...

The screenshot shows a search engine interface with the query 'string matching algorithm'. The results include a Wikipedia entry titled 'String-searching algorithm - Wikipedia' and a link to 'String Matching Algorithms - School of Computer Science ...'. The Wikipedia entry text mentions 'matching strategies' and 'string-searching algorithms', and the school link text mentions 'String matching or searching algorithms try to find places where one or several strings (also called patterns) are found within a larger string (searched text)'.

► Finding similar DNA sequences

- DNA is a long sequence of characters {A, C, G, T}
- Human DNA: a sequence of 3 billion characters

A C T C G C A A T A T G C T A G G C C A G C

A C T _ _ _ _ T T A T G C T A T G C _ _ G C

A C T T G T C T T A T G C

A C T _ G _ _ T T A _ _ C

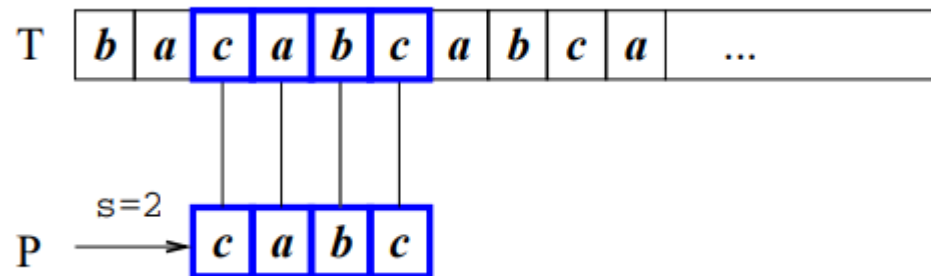




String matching problem

► Problem settings

- A text string array $T[1..n]$
- A pattern string array $P[1..m]$
- All the elements of T and P are taken from an alphabet
- Goal: find all the occurrences of P in T
 - A pattern P occurs with shifts s in T , if $P[1..m] = T[s+1..s+m]$
 - The problem above is to find all values of s ($0 \leq s \leq n-m$)





Concepts: prefix and suffix

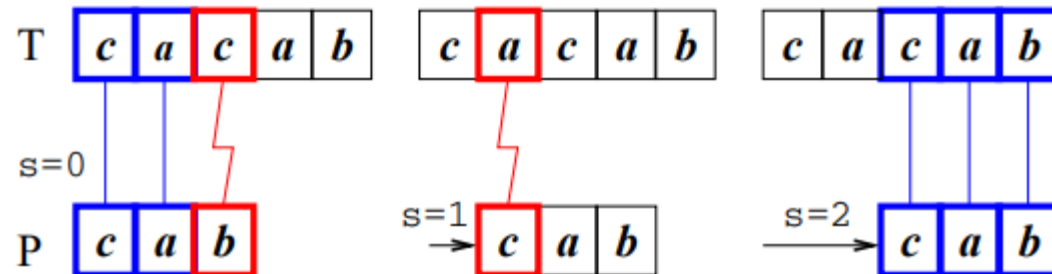
- ▶ A string w is a **prefix** of x , if $x = wy$, for some string y
 - E.g., "**abc**" is a prefix of "**abc**def"
- ▶ Similarly, a string w is a **suffix** of x , if $x = yw$, for some string y
 - E.g., "**abc**" is a suffix of "uvw**abc**"



A brute force algorithm

▶ Algorithm steps

- Initially, **P** is aligned with **T** at the first index position
- **P** is then compared with **T** from left-to-right
- If a mismatch occurs, slide **P** to right by 1 position and start the comparison again





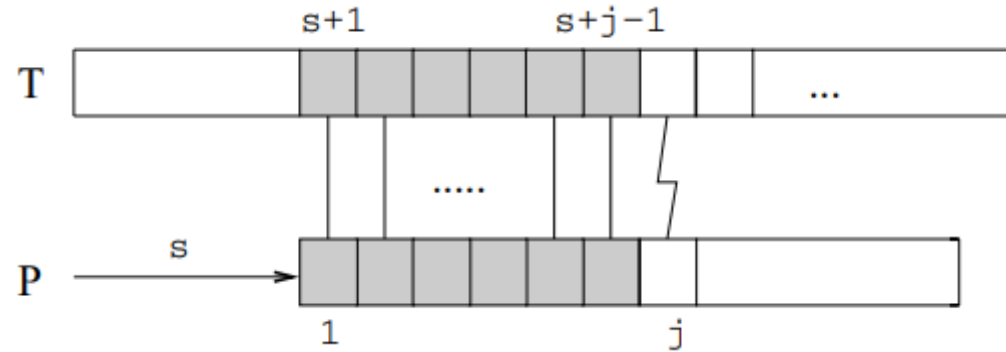
A brute force algorithm

```
BF_StringMatcher(T, P) {  
    n = length(T);  
    m = length(P);  
  
    // s increments by 1 in each iteration  
    // => slide P to right by 1  
    for (s=0; s<=n-m; s++) {  
        // starts the comparison of P and T again  
        i=1; j=1;  
        while (j<=m && T[s+i]==P[j]) {  
            // corresponds to compare P and T from  
            // left-to-right  
            i++; j++;  
        }  
        if (j==m+1)  
            print "Pattern occurs with shift=", s  
    }  
}
```

Time complexity: $O(mn)$



A brute force algorithm



► Limitations

- If a mismatch occurs at $P[j]$ ($j > 1$), it only slides P to right by 1 step
- It throws away one piece of information that we've known
 - Let s be the current shift value
 - Since it is a mismatch at $P[j]$, we know $T[s+1\dots s+j-1] = P[1\dots j-1]$

► How to make use of this information?

- P should slide by $s' > s$ such that $P[1\dots k] = T[s'+1\dots s'+k]$, and then we compare $P[k+1]$ with $P[s'+k+1]$



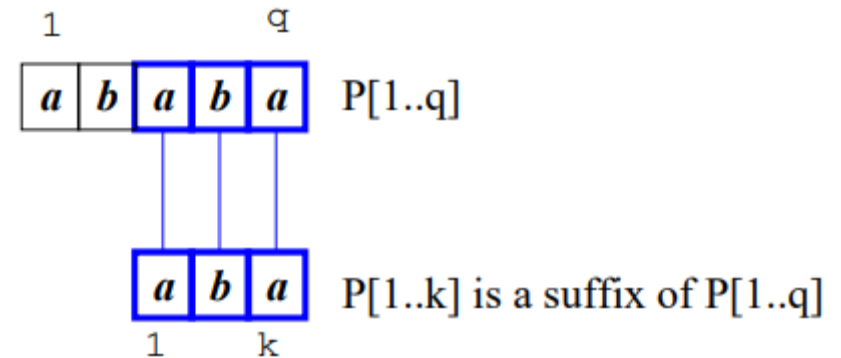
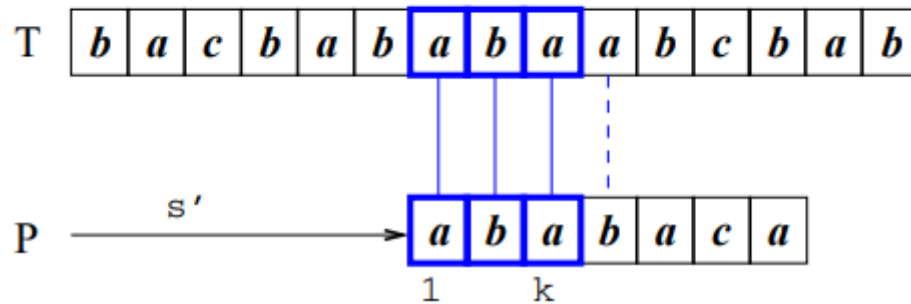
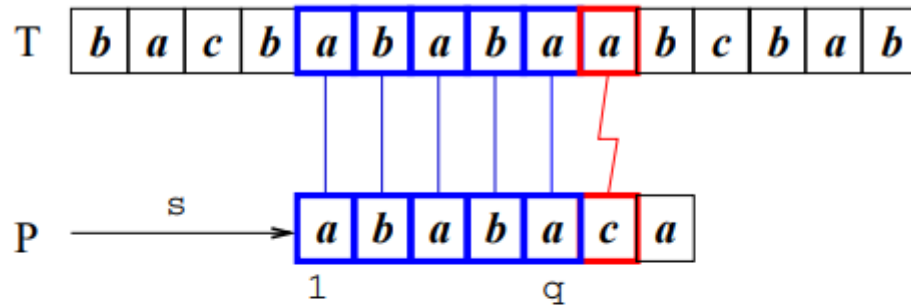
KMP algorithm

- ▶ Knuth-Morris-Pratt (KMP) algorithm
 - It was conceived by Donald Knuth and Vaughan Pratt, and independently by James H. Morris in 1977
 - It keeps the information that brute force algorithm wasted during text scanning, and its time cost is reduced to $O(m + n)$ by using large shifts
 - The implementation of KMP algorithm is efficient because it minimizes the total number of comparisons of the pattern against the input string



KMP algorithm

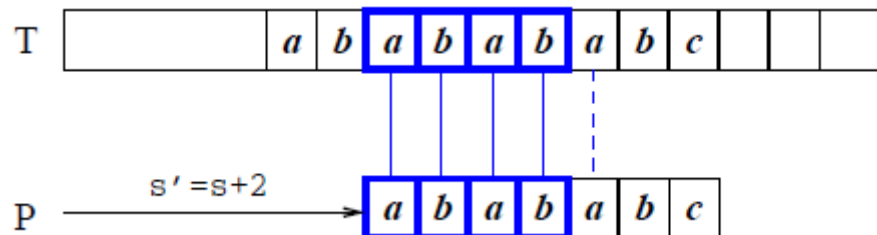
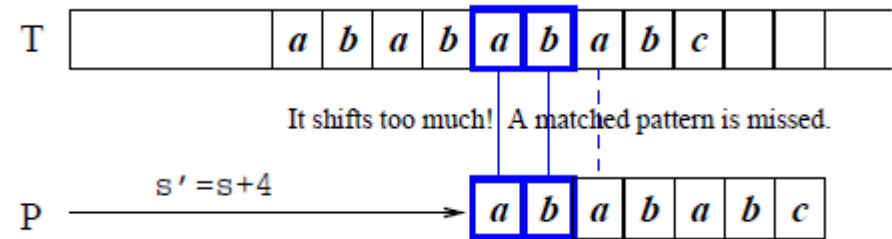
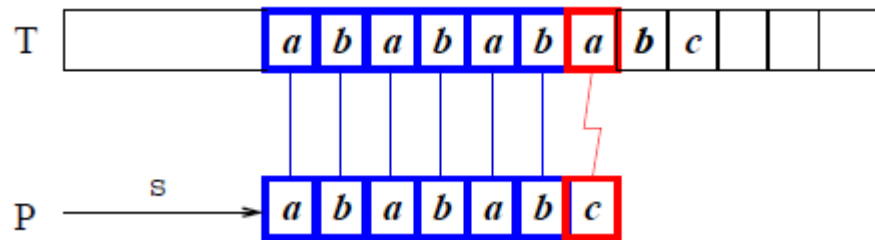
- When we slide P to right, it should be at a place where P could possibly occur in T





KMP algorithm

- Do not shift too much, because it may miss some matched patterns





KMP algorithm

Full example:

$T[] = \text{xyabctxabctxadcdqfeg}$

$P[] = \text{abctxabcde}$

Assume that we have already shifted past the first two positions in T

xyabctxabctxadcdqfeg

abctxabcde

^

1 start again from position 4



KMP algorithm

- ▶ A key question:

Given that $P[1...q]$ match text characters $T[s+1...s+q]$,
what is the least shift $s' > s$ such that
 $P[1...k] = T[s'+1...s'+k]$, where $s'+k = s+q$?



The **next** function

- ▶ Observe that $T[s'+1\dots s'+k]$ is a known text, and it is a suffix of $P[1\dots q]$, so the shift s' can be pre-computed by comparing P against itself
- ▶ To find the least shift $s' > s$, it is the same as finding the largest $k < q$, such that
 $P[1\dots k]$ is a suffix of $P[1\dots q]$
- ▶ Given $P[1\dots m]$, let **next** be a function $\{1, 2, \dots, m\} \rightarrow \{0, 1, \dots, m-1\}$ such that

$$\text{next}(q) = \max\{ k: k < q \text{ and } P[1\dots k] \text{ is a suffix of } P[1\dots q] \}$$

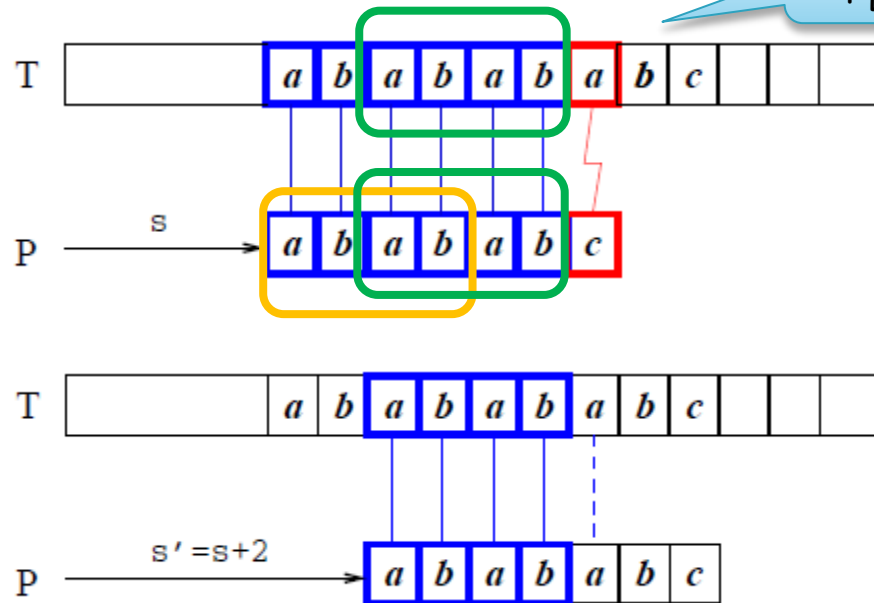


Example

$\text{next}(q) = \max\{ k: k < q \text{ and } P[1..k] \text{ is a suffix of } P[1..q] \}$

q	1	2	3	4	5	6	7	8	9	10
$P[q]$	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>a</i>
$\text{next}(q)$	0	0	1	2	3	4	5	6	0	1

Since $P[1..4]$ is a suffix of $P[1..6]$, we have $\text{next}[6] = 4$





KMP algorithm

- ▶ KMP shift rule:
 - Mismatch case
 - Shift P to the right, $q = \text{next}[q]$
 - Match case
 - If no mismatch is found, an occurrence of P has been found
 - Shift P by $\text{next}[m]$ spaces to continue searching for other occurrences

Question: if all the characters of P are distinct, will KMP algorithm beat the brute force algorithm?



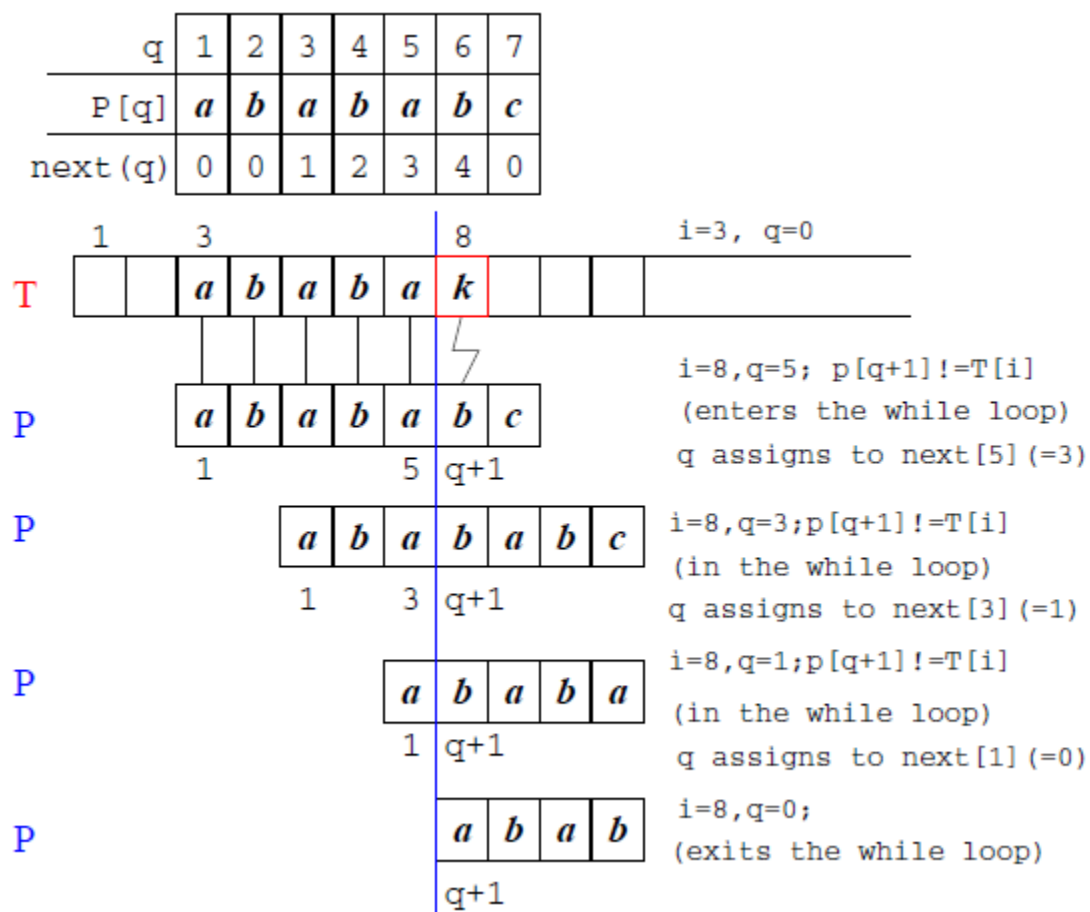
The full KMP algorithm

```
KMP_StringMatcher(T, P) {  
    n = length(T); m = length(P);  
    compute_Next(P);  
    q = 0; // number of characters matched  
           // so far  
    i=1;  
    while (i<=n) {  
        // loop until a match is found, or  
        // number of characters matched so far  
        // is 0; note 'i' is unchanged.  
        while (q > 0 and P[q+1] != T[i]) {  
            q=next[q];  
        }  
        // matched character increased by 1  
        if (P[q+1]==T[i]) q=q+1;  
        if (q==m) {  
            print "Pattern occurs with shift=", i-m  
            q=next[q];  
        }  
        i++;  
    }  
}
```

Time cost: $O(m+n)$ by amortized analysis



A running example





How to compute **next** function

$\text{next}(q) = \max\{ k: k < q \text{ and } P[1\dots k] \text{ is a suffix of } P[1\dots q] \}$

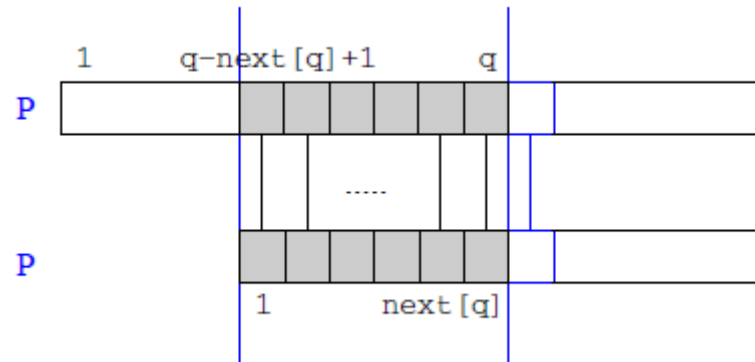
- ▶ A naïve approach
 - Consider each substring $P[1\dots q]$, where q is in $[1\dots m]$
 - For each prefix string $P[1\dots k]$, where k is in $[1\dots m)$, we verify whether it is a suffix string of $P[1\dots q]$, and take the largest qualified k values
 - Time cost: $O(m^3)$

q	1	2	3	4	5	6	7	8	9	10
$P[q]$	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>a</i>
$\text{next}(q)$	0	0	1	2	3	4	5	6	0	1



How to compute **next** function

- ▶ Given $\text{next}[1], \text{next}[2], \dots, \text{next}[q]$, how can we compute $\text{next}[q+1]$?
- ▶ Case 1:
 - If $P[q+1] = P[\text{next}[q]+1]$, then $\text{next}[q+1] = \text{next}[q]+1$

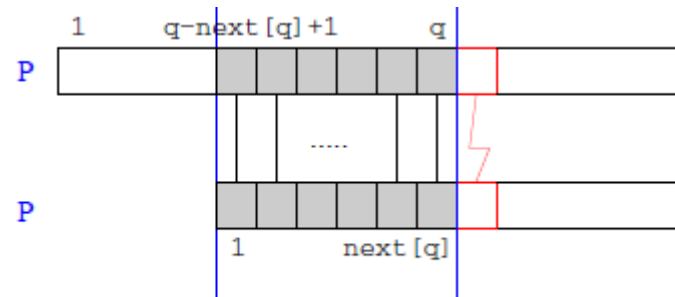




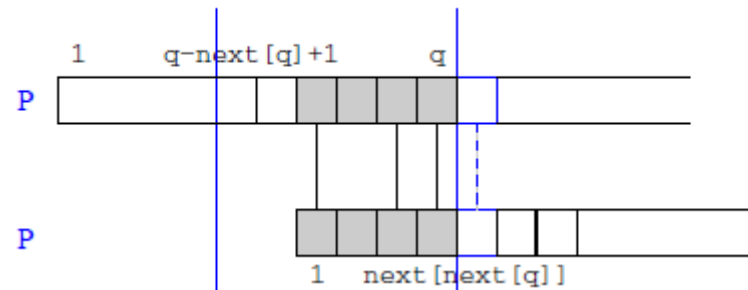
How to compute **next** function

► Case 2:

- If $P[q+1] \neq P[\text{next}[q]+1]$, then do what?
- P should slide to a place such that the prefix of $P[1..\text{next}[q]]$ occurs as a suffix of $P[q-\text{next}[q]+1..q]$; this information is stored in $\text{next}[\text{next}[q]]$!



observe that $P[1..\text{next}[q]] = P[q-\text{next}[q]+1..q]$





How to compute **next** function

- ▶ We first set $\text{next}[1] = 0$, then compute $\text{next}[q]$ with $q = 2, 3, \dots, m$, one by one in $(m - 1)$ iterations

```
compute_Next(P) {  
    m = length(P);  
    next[1]=0; // initialization  
    k = 0; // number of characters matched  
           // so far  
    q=2;  
    while (q<=m) {  
        while (k > 0 and P[k+1] != P[q]) {  
            k = next[k];  
        }  
        if (P[k+1]==P[q]) k=k+1;  
        next[q]=k;  
  
        q++;  
    }  
}
```

Time cost: $O(m)$ by amortized analysis



String Hashing

- ▶ String Hashing

$$f(s) = \sum_{i=1}^l s[i] \times b^{l-i} \pmod{M}$$

- ▶ For xyz, $f(\text{xyz}) = x \cdot b^2 + y \cdot b + z$

$$f(s[l..r]) = s[l] \cdot b^{r-l} + s[l+1] \cdot b^{r-l-1} + \dots + s[r-1] \cdot b + s[r]$$

$$f(s[l..r]) = f_r(s) - f_{l-1}(s) \times b^{r-l+1}$$



Summary

- ▶ KMP is used for string matching after preprocessing.
- ▶ String Hashing can be wrong but usually more applicable under multiple scenarios (and maybe easier to implement)
- ▶ But you should know both!!!



Palindrome Finding

- ▶ How to represent all palindromes in a string?
- ▶ For each center, we find its palindromic radius.
- ▶ How to use string hashing to find it? What's the complexity?



Manacher

- ▶ We describe the algorithm to find all the sub-palindromes with odd length, i. e. to calculate $d_{\text{odd}}[]$.
- ▶ For fast calculation we'll maintain the **exclusive borders** (l, r) of the rightmost found (sub-)palindrome (i. e. the current rightmost (sub-)palindrome is $s[l+1] s[l+2] \dots s[r-1]$). Initially we set $l = 0, r = 1$, which corresponds to the empty string.



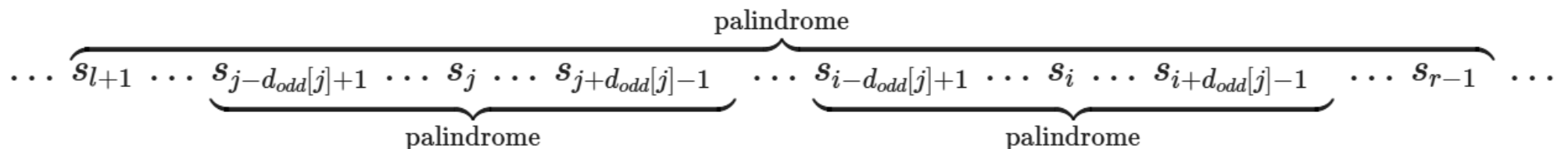
Manacher

- ▶ So, we want to calculate $d_{\text{odd}}[i]$ for the next i , and all the previous values in $d_{\text{odd}}[]$ have been already calculated. We do the following:
- ▶ If i is outside the current sub-palindrome, i. e. $i \leq r$, we'll just launch the trivial algorithm.
- ▶ So we'll increase $d_{\text{odd}}[i]$ consecutively and check each time if the current rightmost substring $[i - d_{\text{odd}}[i].. i + d_{\text{odd}}[i]]$ is a palindrome. When we find the first mismatch or meet the boundaries of s , we'll stop.
- ▶ In this case we've finally calculated $d_{\text{odd}}[i]$. After this, we must not forget to update (l, r) . r should be updated in such a way that it represents the last index of the current rightmost sub-palindrome.



Manacher

- ▶ Now consider the case when $i \leq r$. We'll try to extract some information from the already calculated values in $d_{\text{odd}}[]$. So, let's find the "mirror" position of i in the sub-palindrome (l, r) , i.e. we'll get the position $j = l + (r - i)$, and we check the value of $d_{\text{odd}}[j]$.
- ▶ Because j is the position symmetrical to i with respect to $(l+r)/2$, we can **almost always** assign $d_{\text{odd}}[i] = d_{\text{odd}}[j]$. Illustration of this (palindrome around j is actually "copied" into the palindrome around i):





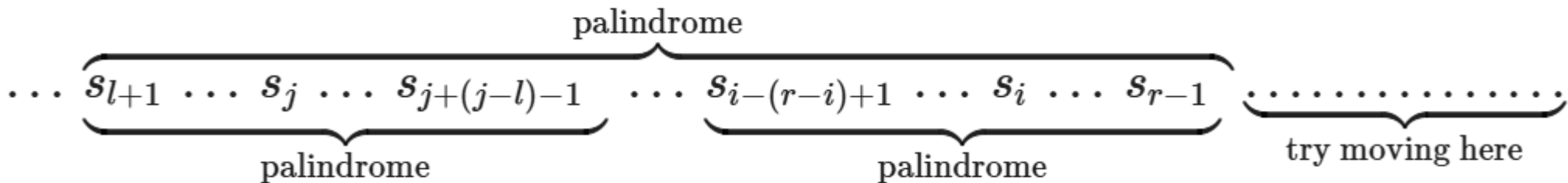
Manacher

- ▶ But there is a **tricky case** to be handled correctly: when the "inner" palindrome reaches the borders of the "outer" one, i. e. $j - d_{\text{odd}}[j] \leq l$ (or, which is the same, $i + d_{\text{odd}}[j] \geq r$).
- ▶ Because the symmetry outside the "outer" palindrome is not guaranteed, just assigning $d_{\text{odd}}[i] = d_{\text{odd}}[j]$ will be incorrect: we do not have enough data to state that the palindrome in the position i has the same length.



Manacher

- ▶ Actually, we should restrict the length of our palindrome for now, i. e. assign $d_{\text{odd}}[i] = r - i$, to handle such situations correctly. After this we'll run the trivial algorithm which will try to increase $d_{\text{odd}}[i]$ while it's possible.
- ▶ (the palindrome with center j is restricted to fit the "outer" palindrome)
- ▶ Again, we should not forget to update the values (l, r) after calculating each $d_{\text{odd}}[i]$.





Complexity

```
1  d1 = [0] * n
2  l, r = 0, -1
3  for i in range(0, n):
4      k = 1 if i > r else min(d1[l + r - i], r - i + 1)
5      while 0 <= i - k and i + k < n and s[i - k] == s[i + k]:
6          k += 1
7      d1[i] = k
8      k -= 1
9      if i + k > r:
10         l = i - k
11         r = i + k
```

- ▶ We can notice that every iteration of trivial algorithm increases r by one. Also r cannot be decreased during the algorithm. So, trivial algorithm will make $O(n)$ iterations in total.



Reading if you are so confused.

- ▶ Manacher's Algorithm - Finding all sub-palindromes in $O(N)$ - Algorithms for Competitive Programming
- ▶ Manacher - OI Wiki
- ▶ You may not need to memorize the whole thing, but must know what's going on with this!



Summary

- ▶ We learn a string matching algorithm KMP and a palindrome decomposition algorithm Manacher
- ▶ We also learn the string hashing technique that can serve as an alternative method above,



Palindrome Dividing

- ▶ Given a string, find out the minimum cut to make each portion a palindrome.
- ▶ $N \leq 1000$